

**LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK**

PRAKTIKUM 4
"EKSPANSI PRODUK KEUANGAN (INHERITANCE)"

disusun Oleh:
DENNIS SHAUQI AKBAR
2511533020

Dosen Pengampu:
Nurfiah, S.ST, M.Kom

Asisten Praktikum:
Siti Fadhilah Rahmi



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026**

LAPORAN PRAKTIKUM 4

"EKSPANSI PRODUK KEUANGAN (INHERITANCE)"

A. Tahapan Pelaksanaan Praktikum

Langkah 1: Memastikan Akses Modifier di Rekening.java

```
1 package pekan4;
2 import java.util.*;
3
4 public class Rekening {
5     // Bagian atas dari class Rekening.java
6     private String nomorRekening;
7     private String namaPemilik;
8     private String pin;
9
10    // Gunakan protected agar subclass bisa mengaksesnya langsung
11    protected double saldo;
12    protected ArrayList<Transaksi> riwayatTransaksi;
13
14
15    //... (constructor dan method lainnya tetap sama seperti Modul 3)
16
```

Pada langkah ini class Rekening disiapkan sebagai superclass (kelas induk). Atribut nomorRekening, namaPemilik, dan pin tetap dibiarkan private karena bersifat sensitif dan tidak perlu diakses oleh kelas turunan. Sebaliknya, atribut saldo dan riwayatTransaksi diubah menjadi protected agar dapat diakses langsung oleh subclass (RekeningTabungan dan RekeningGiro) tanpa harus dibuka ke akses publik. Dengan modifier protected, atribut hanya bisa diakses oleh class dalam package yang sama serta oleh subclass, sehingga enkapsulasi tetap terjaga sekaligus memberi ruang bagi pewarisan.

Langkah 2: Membuat Subclass RekeningTabungan

```

package pekan4;

public class RekeningTabungan extends Rekening {
    //Atribut Spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    //Constructor Subclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {
        // super() memanggil constructor kelas induk (Rekening). WAJIB berada di baris pertama!
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        // Menghitung bunga
        // mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan ?
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        // Mencatat Riwayat Transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp" + nominalBunga);
    }
}

```

Class RekeningTabungan dibuat sebagai subclass dari Rekening menggunakan keyword extends, sehingga otomatis mewarisi seluruh atribut dan method milik induknya (relasi Is-A: Tabungan adalah sebuah Rekening). Class ini menambahkan satu atribut khusus, yaitu sukuBunga yang bersifat private. Pada constructor, pemanggilan super(nomor, nama, saldoAwal, pinAwal) berfungsi mengeksekusi constructor superclass untuk menginisialisasi data dasar rekening, dan wajib berada di baris pertama constructor. Baru setelah itu atribut sukuBunga diisi. Method tambahBungaAkhirBulan() menghitung bunga dari saldo saat ini dikalikan suku bunga dalam persen, menambahkannya ke saldo, lalu mencatat transaksi berjenis "Bunga" dengan ID berawalan "TRX-B-" ke dalam riwayatTransaksi.

Method tambahBungaAkhirBulan() dapat mengakses saldo dan riwayatTransaksi secara langsung karena kedua atribut tersebut dideklarasikan protected di superclass Rekening, sehingga dapat diakses oleh subclass tanpa memerlukan getter atau setter.

Output pengujian:

Pembuatan rekening bertipe Tabungan melalui menu 1 dengan memilih produk 1 (Tabungan Umum) dan suku bunga 5%:

=== SISTEM PERBANKAN MINI ===

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 1

Masukkan No Rekening: 39284

Masukkan Nama Pemilik: dennis

Masukkan Saldo Awal: 100000000

Masukkan Pin (6 digit): 212223

Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis

Pilihan: 1

Masukkan Suku Bunga (%): 5

Rekening atas nama dennis berhasil dibuat

Rekening berhasil ditambahkan. Total rekening: 1

Program berhasil membuat objek RekeningTabungan atas nama dennis dengan nomor rekening 39284 dan saldo awal Rp100.000.000. Pesan "Rekening atas nama dennis berhasil dibuat" berasal dari constructor superclass Rekening yang dipanggil melalui `super()`, membuktikan bahwa inisialisasi induk berjalan saat subclass dibuat.

Langkah 3: Membuat Subclass RekeningGiro

```
package pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal, double batasOverdraft) {
        // Memanggil inisialisasi dasar dari SuperClass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
    // (Catatan: Penarikan hingga limit Overdraft akan kita selesaikan di Modul 5)
}
```

Class `RekeningGiro` juga merupakan subclass dari `Rekening` yang dibuat dengan `extends`. Fitur tambahannya adalah `batasOverdraft` (limit pinjaman) yang bersifat `private` dan disediakan getter `getBatasOverdraft()` agar nilainya bisa dibaca dari luar class. Constructor-nya memanggil `super(...)` di baris pertama untuk menginisialisasi data dasar rekening, kemudian mengisi atribut `batasOverdraft`. Logika penarikan hingga batas

overdraft belum diimplementasikan pada modul ini dan akan diselesaikan pada modul berikutnya.

Output pengujian:

Pembuatan rekening bertipe Giro melalui menu 1 dengan memilih produk 2 (Giro Bisnis) dan batas overdraft Rp500.000:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 238473
Masukkan Nama Pemilik: dennis_3020
Masukkan Saldo Awal: 15000000
Masukkan Pin (6 digit): 199193
Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis
Pilihan: 2
Masukkan Batas Overdraft: 500000
Rekening atas nama dennis_3020 berhasil dibuat
Rekening berhasil ditambahkan. Total rekening: 2
```

Program berhasil membuat objek RekeningGiro atas nama dennis_3020 dan menambahkannya ke daftar rekening sehingga total rekening menjadi 2. Hal ini menunjukkan bahwa dua jenis produk rekening yang berbeda dapat disimpan dalam satu koleksi yang sama.

B. Latihan/Tugas

Tugas 1: Pembuatan Multi-Produk

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();
    System.out.print("Masukkan Pin (6 digit): ");
    String pin = input.nextLine();

    System.out.println("Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis");
    System.out.print("Pilihan: ");
    int produk = Integer.parseInt(input.nextLine());

    Rekening baru; // tipe Superclass

    if (produk == 1) {
        System.out.print("Masukkan Suku Bunga (%): ");
        double bunga = Double.parseDouble(input.nextLine());
        baru = new RekeningTabungan(no, nama, saldo, pin, bunga); // Upcasting
    } else if (produk == 2) {
        System.out.print("Masukkan Batas Overdraft: ");
        double limit = Double.parseDouble(input.nextLine());
        baru = new RekeningGiro(no, nama, saldo, pin, limit); // Upcasting
    } else {
        System.out.println("Pilihan produk tidak valid.");
        break;
    }

    akunAktif = baru;
    rekeningAktif.add(baru);
    System.out.println("Rekening berhasil ditambahkan. Total rekening: " + rekeningAktif.size());
    break;
```

Pada case 1, setelah pengguna mengisi nomor rekening, nama, saldo awal, dan PIN, program menampilkan pilihan produk. Variabel baru dideklarasikan bertipe Rekening (superclass). Jika pengguna memilih produk 1, program meminta suku bunga lalu membuat objek RekeningTabungan; jika memilih produk 2, program meminta batas overdraft lalu membuat objek RekeningGiro; jika memilih selain itu, program menampilkan pesan bahwa pilihan tidak valid dan keluar dari case. Objek subclass yang disimpan ke dalam variabel bertipe Rekening merupakan upcasting yang terjadi secara implisit dan aman. Berkat upcasting ini, objek Tabungan maupun Giro dapat disimpan bersama dalam satu ArrayList<Rekening>.

Output pengujian:

Membuat rekening Tabungan Umum dengan suku bunga 5%:

=== SISTEM PERBANKAN MINI ===

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 1

Masukkan No Rekening: 39284

Masukkan Nama Pemilik: dennis

Masukkan Saldo Awal: 100000000

Masukkan Pin (6 digit): 212223

Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis

Pilihan: 1

Masukkan Suku Bunga (%): 5

Rekening atas nama dennis berhasil dibuat

Rekening berhasil ditambahkan. Total rekening: 1

Membuat rekening Giro Bisnis dengan batas overdraft Rp500.000:

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 1

Masukkan No Rekening: 238473

Masukkan Nama Pemilik: dennis_3020

Masukkan Saldo Awal: 15000000

Masukkan Pin (6 digit): 199193

Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis

Pilihan: 2

Masukkan Batas Overdraft: 500000

Rekening atas nama dennis_3020 berhasil dibuat

Rekening berhasil ditambahkan. Total rekening: 2

Kedua pengujian berhasil, dan total rekening bertambah dari 1 menjadi 2, membuktikan bahwa dua jenis subclass berbeda dapat disimpan dalam satu koleksi bertipe superclass.

Tugas 2: Simulasi Waktu Akhir Bulan

```
case 7:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else if (akunAktif instanceof RekeningTabungan) {
        RekeningTabungan tab = (RekeningTabungan) akunAktif; // Downcasting
        tab.tambahBungaAkhirBulan();
    } else {
        System.out.println("Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.");
    }
    break;
```

Menu baru "7. Simulasi Akhir Bulan (Khusus Tabungan)" ditambahkan pada menu utama. Pada case 7, program memeriksa tiga kondisi secara berurutan. Pertama, apakah akunAktif masih null. Kedua, apakah akunAktif adalah objek RekeningTabungan menggunakan operator instanceof. Jika ya, dilakukan downcasting eksplisit ke tipe RekeningTabungan agar method khusus tambahBungaAkhirBulan() dapat dipanggil, karena method tersebut tidak dikenal oleh tipe Rekening. Ketiga, jika akun bukan Tabungan, program menolak dengan pesan kegagalan. Pengecekan instanceof sebelum downcasting penting untuk mencegah ClassCastException saat objek aslinya ternyata bukan Tabungan.

Output pengujian:

Percobaan pertama, memilih menu 7 saat akun aktif adalah rekening Giro (akun yang dibuat paling akhir):

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.
```

Program menolak dengan pesan kegagalan karena hasil instanceof bernilai false untuk objek RekeningGiro, sehingga downcasting tidak dilakukan dan tidak terjadi error.

Selanjutnya dilakukan pergantian akun aktif ke rekening Tabungan (nomor 39284) melalui menu 5:

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 5

Masukkan nomor rekening yang ingin diaktifkan: 39284

Berhasil! Akun aktif sekarang atas nama dennis (No. Rekening: 39284)

Percobaan kedua, memilih menu 7 saat akun aktif adalah rekening Tabungan dengan saldo Rp100.000.000 dan suku bunga 5%:

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 7

Bunga 5.0% berhasil ditambahkan: Rp5000000.0

Bunga berhasil ditambahkan sebesar Rp5.000.000, sesuai perhitungan 5% dari saldo Rp100.000.000. Ini membuktikan bahwa downcasting berhasil dan method khusus milik RekeningTabungan dapat dijalankan.

Pengecekan riwayat transaksi menggunakan menu 6 dengan PIN yang benar:

Menu Utama:

1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar

Pilih menu: 6

Masukkan PIN: 212223

ID: TRX-B-1790594447084 | Jenis: Bunga | Nominal: Rp5000000.0

Riwayat transaksi mencatat satu transaksi berjenis Bunga dengan ID berawalan TRX-B- , yang menunjukkan bahwa subclass berhasil menulis ke riwayatTransaksi milik superclass Rekening.

Pengecekan saldo akhir menggunakan menu 4:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 39284
Nama Pemilik : dennis
Saldo Akhir  : Rp1.05E8
-----
```

Saldo akhir tampil sebagai Rp1.05E8. Ini adalah cara Java menampilkan bilangan bertipe double yang bernilai besar dalam notasi ilmiah, dengan 1.05E8 sama dengan $1,05 \times 10^8$ atau Rp105.000.000. Nilai ini sesuai dengan saldo awal Rp100.000.000 ditambah bunga Rp5.000.000.

C. Link Repository GitHub

https://github.com/shauqiakbar/PrakPBO_2511533020/tree/pekan4